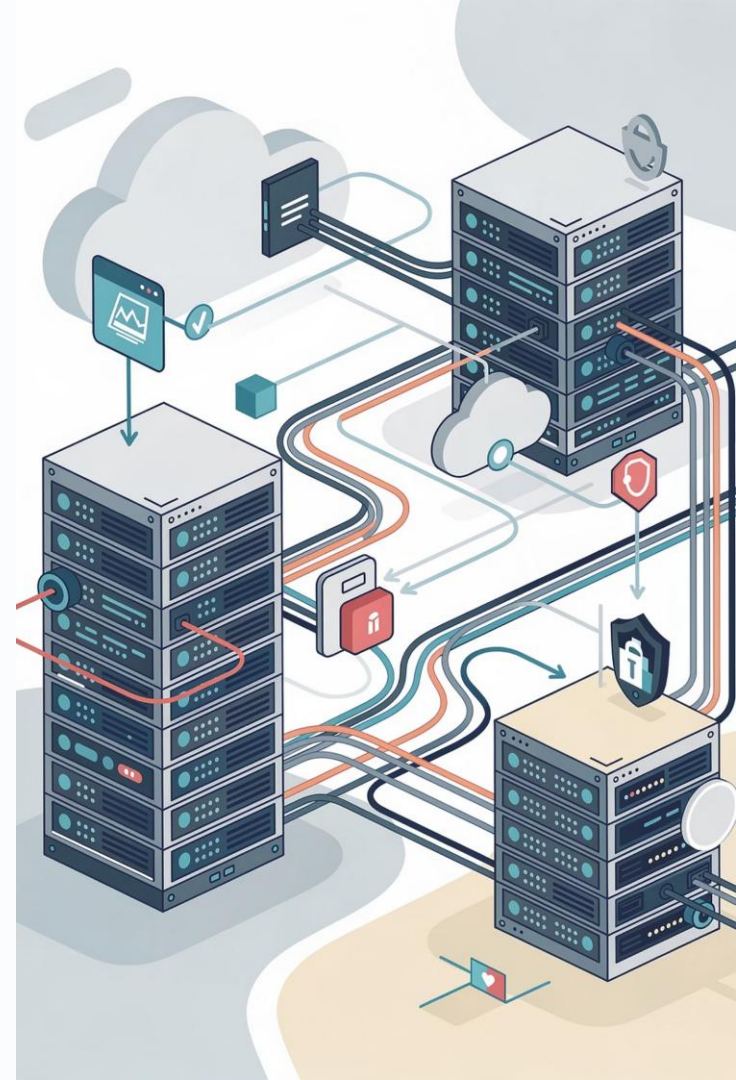


Desarrollo Web

Introducción a la Programación 2



Al finalizar esta sesión, podrás...

1

Explicar la Web

Describir cómo funciona Internet y la arquitectura cliente-servidor en términos técnicos y prácticos.

2

Front End vs Back End

Diferenciar con claridad las capas de presentación y lógica de negocio en una aplicación web.

3

Protocolos Web

Comprender HTTP, HTTPS y FTP: sus diferencias, propósitos y casos de uso reales.

4

HTML, CSS y JS

Identificar los fundamentos de las tecnologías base del desarrollo web moderno.

5

JSON y XML

Reconocer los principales formatos de intercambio de datos y su rol en aplicaciones web.



ACTIVIDAD INICIAL 

¿Qué ocurre cuando escribes `www.google.com` y presionas Enter?

Tómate 2 minutos para pensar en la respuesta. Después, escríbela en el chat del videoconferencia. ¡No hay respuesta incorrecta en este momento!

Tu respuesta en el chat

Escribe en 1-2 oraciones qué crees que sucede técnicamente desde que presionas Enter hasta que ves la página.

Lo que exploraremos hoy

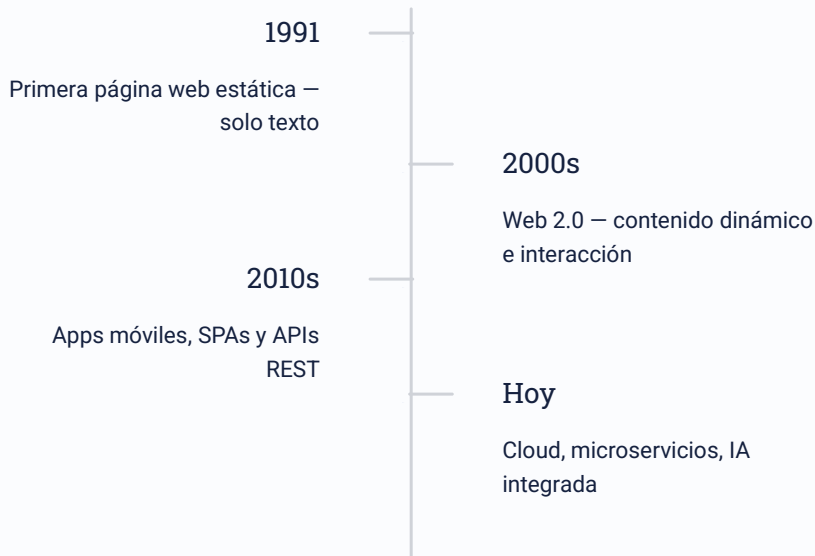
DNS, servidores, HTTP, renderizado, bases de datos, APIs... ¡todo está conectado y lo veremos paso a paso!

 Esta pregunta tiene respuesta: al finalizar la clase, podrás explicarla con precisión técnica. ¡Guarden su respuesta inicial para comparar al final!

¿Qué es el Desarrollo Web?

El **desarrollo web** es el proceso de crear, construir y mantener sitios y aplicaciones accesibles a través de Internet. Abarca desde páginas estáticas simples hasta plataformas complejas con millones de usuarios simultáneos.

Evolución histórica



Aplicaciones web que usas cada día



Netflix

Streaming con recomendaciones en tiempo real



Facebook / Meta

Red social con feeds personalizados y IA



Amazon

E-commerce con logística y pagos integrados

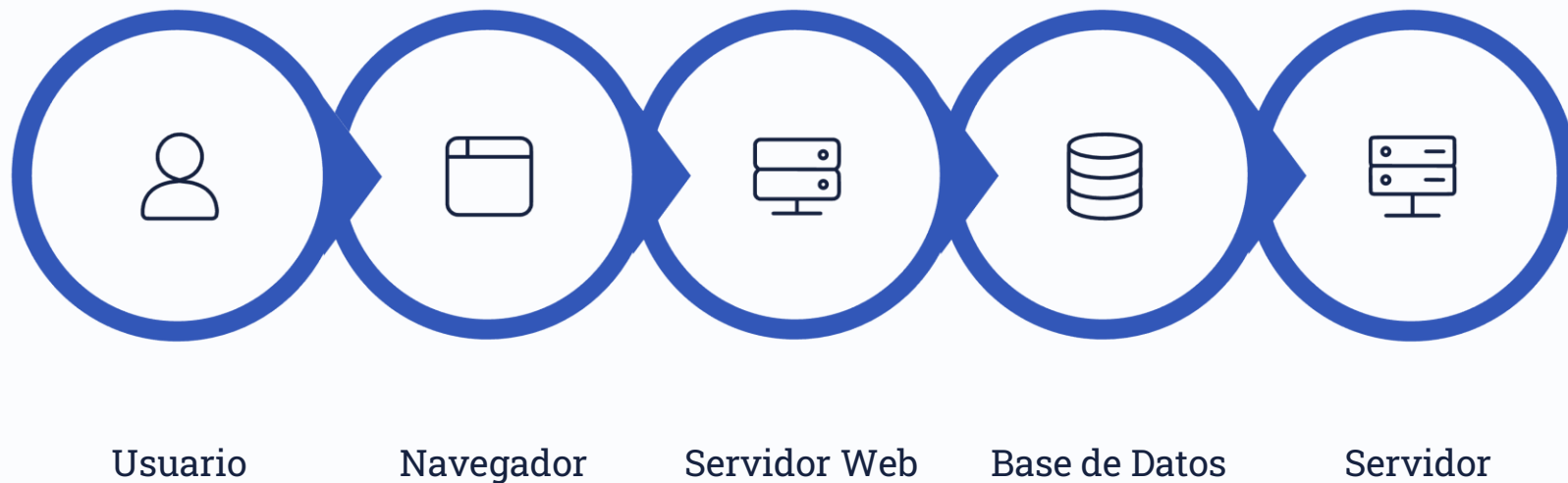


Spotify

Música en streaming con algoritmos adaptativos

El Viaje de una Petición Web

Cada vez que visitas un sitio web, ocurre un flujo completo de comunicación entre múltiples actores tecnológicos. Este es el ecosistema completo:



Este ciclo ocurre en **milisegundos**. Cada componente cumple una función específica e irremplazable dentro del ecosistema. A lo largo de esta sesión, analizaremos cada uno de estos actores en detalle.

¿Qué es un Servidor Web?

Definición

Un servidor web es un **programa (software)** que **escucha peticiones** de clientes a través de la red y les responde enviando recursos: páginas HTML, imágenes, archivos, datos JSON, etc. También puede referirse al hardware (computador) donde ese software está instalado.

La analogía del restaurante 🍽️

El **cliente** es el comensal que ordena. El **navegador** es el mesero que lleva la orden. El **servidor** es la cocina que prepara y envía el plato. La **base de datos** es la despensa de ingredientes.

Servidores web más usados

Apache

El más antiguo y popular. Open source. Usa `.htaccess` para configuración.

Nginx

Alta performance, ideal para tráfico masivo. Usado por Netflix y Airbnb.

IIS

Internet Information Services de Microsoft. Integrado en entornos Windows Server.

- 📘 **Solicitud** → El cliente envía un mensaje HTTP al servidor.
- Respuesta** → El servidor retorna los recursos solicitados con un código de estado (200, 404, 500...).

¿Quién realiza cada tarea?

Clasifica cada acción según el componente responsable: **Navegador**, **Servidor** o **Base de Datos**. Responde en el chat.



Navegador

- Renderizar el HTML/CSS en pantalla
- Ejecutar código JavaScript
- Almacenar cookies y caché
- Enviar la petición HTTP



Servidor

- Procesar la lógica de negocio
- Autenticar al usuario
- Consultar y escribir en la BD
- Generar y enviar la respuesta



Base de Datos

- Almacenar registros de usuarios
- Retornar resultados de búsquedas
- Guardar transacciones
- Mantener relaciones entre datos

❓ Mini reto: ¿Dónde ocurre la validación de un formulario de registro? ¿Solo en el navegador, solo en el servidor, o en ambos? ¿Por qué?

¿Cómo funciona un Navegador Web?

Los navegadores más populares



Chrome

El más usado a nivel mundial. Motor V8 para JavaScript.



Firefox

Open source, enfocado en privacidad. Motor Gecko.



Edge

De Microsoft, basado en Chromium. Integrado en Windows.



Safari

Nativo de Apple. Optimizado para macOS e iOS.

Proceso interno del navegador



1. URL

El usuario ingresa una dirección. El navegador la interpreta y extrae: protocolo, dominio y ruta.



2. DNS

El Sistema de Nombres de Dominio traduce `google.com` a una dirección IP como `142.250.80.46`.



3. Renderizado

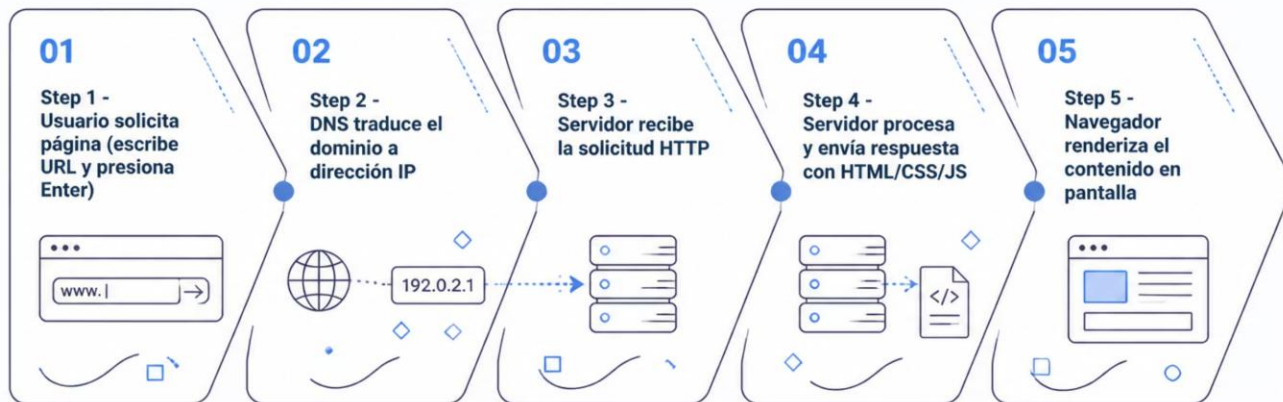
El motor del navegador parsea HTML, aplica CSS y ejecuta JS para construir el DOM visible.



4. Caché

Guarda localmente recursos para acelerar futuras visitas y reducir tráfico de red.

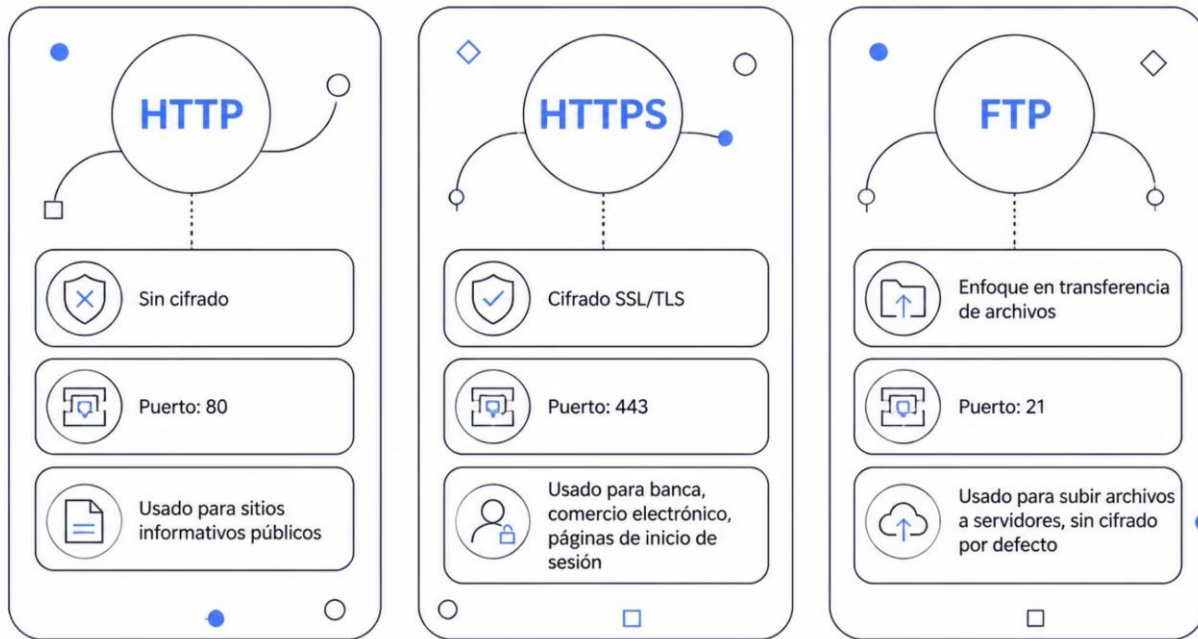
Del clic del usuario a la página renderizada



Este proceso completo, que involucra resolución DNS, establecimiento de conexión TCP, handshake TLS (en HTTPS), transferencia de datos y renderizado, ocurre típicamente en **menos de 500 milisegundos**. La optimización de cada etapa es clave en el desarrollo web profesional.

HTTP, HTTPS y FTP

Los protocolos definen las **reglas de comunicación** entre clientes y servidores. Elegir el protocolo correcto es fundamental para la seguridad y funcionalidad de tus aplicaciones.



Hoy en día, **HTTPS es el estándar mínimo** para cualquier sitio web. Los navegadores modernos marcan los sitios HTTP como "No seguros". ¡Nunca desarrolles aplicaciones en producción sin SSL/TLS!

HTTP vs HTTPS en detalle

Característica	HTTP	HTTPS
Cifrado de datos	✗ Sin cifrado (texto plano)	✓ SSL/TLS (cifrado extremo a extremo)
Puerto por defecto	80	443
Certificado digital	No requerido	Obligatorio (CA emisor)
Velocidad	Ligeramente más rápido	Mínima diferencia con HTTP/2
SEO (posicionamiento)	Penalizado por Google	Factor positivo de ranking
Casos de uso	Sitios legacy, redes internas	Todo sitio con login, pagos o datos

ⓘ SSL (Secure Sockets Layer) fue reemplazado por TLS (Transport Layer Security), aunque el término "SSL" se sigue usando coloquialmente. Let's Encrypt ofrece certificados TLS **gratuitos** para cualquier dominio.

¿Qué protocolo usarías?

1

Subir archivos al servidor

Necesitas cargar los archivos de tu proyecto web (HTML, CSS, imágenes) a tu hosting. ¿Qué protocolo usas? **FTP / SFTP** – Herramientas como FileZilla o WinSCP.

2

Banca en línea

El usuario ingresa sus credenciales y realiza una transferencia. Datos sensibles en tránsito. **HTTPS obligatorio** – Certificado EV de alta seguridad.

3

Página informativa pública

Un blog sin login, sin datos personales, solo contenido estático de texto e imágenes. **HTTPS recomendado** – Aunque no haya datos privados, Google penaliza HTTP.

 FTP popular: **FileZilla** (gratuito), **Cyberduck** (macOS), **WinSCP** (Windows). SFTP es la versión segura de FTP – usa el puerto 22 con cifrado SSH.

¿Qué hace atractiva y funcional una página web?



Usabilidad

El usuario logra sus objetivos de forma fácil, intuitiva y eficiente. Navegación clara, botones reconocibles.



Accesibilidad

Diseñada para todos, incluyendo personas con discapacidades visuales, motoras o cognitivas. WCAG 2.1.



Consistencia

Mismos colores, tipografías y patrones de interacción en todas las páginas. Construye confianza y familiaridad.



Diseño Responsivo

La interfaz se adapta fluidamente a cualquier tamaño de pantalla: móvil, tablet, escritorio.



Principios Base de HTML

¿Qué es HTML?

HyperText Markup Language es el lenguaje estándar para estructurar el contenido de las páginas web. Define el **esqueleto y significado semántico** de cada elemento: títulos, párrafos, imágenes, listas, formularios, enlaces.

No es un lenguaje de programación

HTML es un **lenguaje de marcado**: describe la estructura y el significado del contenido, pero no tiene lógica, variables ni condiciones.

Estructura básica y etiquetas esenciales

```
<!DOCTYPE html>
<html lang="es">
  <head>
    <meta charset="UTF-8">
    <title>Mi primera página</title>
  </head>
  <body>
    <h1>Hola, Mundo Web</h1>
    <p>Este es un párrafo de texto.</p>
    
    <a href="https://google.com">Ir a Google</a>
  </body>
</html>
```

✔ Etiquetas clave: `<html>` `<head>` `<body>` `<h1>`-`<h6>` `<p>` `<img>` `<a>`
`<div>` `<span>`

Principios Base de CSS

¿Qué es CSS?

Cascading Style Sheets es el lenguaje que controla la **apariciencia visual** del HTML: colores, tipografías, tamaños, márgenes, animaciones y disposición de los elementos en pantalla.

Sintaxis CSS

```
h1 {  
  color: #3257b8;  
  font-size: 2rem;  
  font-family: 'Arial', sans-serif;  
}  
  
p {  
  color: #333;  
  line-height: 1.6;  
  margin-bottom: 16px;  
}  
  
.boton {  
  display: inline-block;  
  background-color: #3257b8;  
  color: white;  
  padding: 10px 20px;  
  border-radius: 8px;  
  text-decoration: none;  
  font-family: 'Arial', sans-serif;  
  font-weight: bold;  
  transition: background-color 0.3s ease, transform 0.2s ease;  
}  
  
.boton:hover {  
  background-color: #244191;  
  transform: translateY(-2px);  
}
```

Conceptos clave de CSS

Selectores

Indican a *qué elemento* aplicar estilos: por etiqueta (`h1`), clase (`.clase`), ID (`#id`).

Propiedades

`color`, `background`, `font-size`, `margin`, `padding`, `display`, `flex`.

Cascada y Especificidad

Los estilos se aplican en orden de especificidad. ID > Clase > Etiqueta. El último estilo declarado gana si tienen igual peso.

Box Model

Todo elemento es una caja: `content` + `padding` + `border` + `margin`. Fundamental para el layout.



3.2 FRONT END DEVELOPMENT

¿Qué es el Front End?

1

HTML

Estructura y contenido semántico de la página

2

CSS

Estilos visuales, animaciones y diseño responsivo

3

JavaScript

Comportamiento, interactividad y lógica del cliente

El **Front End** es todo lo que el usuario ve, toca e interactúa directamente en su navegador. El desarrollador front end combina estas tres tecnologías para crear **interfaces de usuario intuitivas, accesibles y atractivas** que comunican eficazmente con el back end a través de APIs.

JavaScript: El lenguaje de la interactividad

¿Qué es JavaScript?

JS es el único **lenguaje de programación nativo del navegador**. Permite crear experiencias dinámicas: validar formularios, actualizar el DOM sin recargar la página, manejar eventos del usuario y comunicarse con APIs en tiempo real.

Ejemplos simples

```
// Alerta al usuario
alert("¡Bienvenido a la clase!");

// Manejar evento de clic
document.getElementById("btn")
  .addEventListener("click", function() {
    alert("¡Botón presionado!");
  });

// Manipulación del DOM
document.getElementById("titulo")
  .textContent = "Texto cambiado con JS";
```

¿Para qué sirve JavaScript?

Validaciones

Verificar datos en el formulario antes de enviarlos al servidor.

Eventos

Responder a clicks, teclado, scroll, hover y movimientos del mouse.

DOM dinámico

Crear, modificar o eliminar elementos HTML sin recargar.

APIs y AJAX

Comunicarse con el servidor y actualizar solo partes de la página.



Mini Lab: ¿Qué mostrará este código?

```
let x = 5; let y = 3; alert(x + y + " es el resultado");
```

¿Qué es el Back End?

El motor invisible

El **Back End** es la capa del servidor que el usuario nunca ve pero que hace posible todo. Gestiona la **lógica de negocio**, el acceso a bases de datos, la autenticación de usuarios, el procesamiento de pagos y la seguridad de la aplicación.

Responsabilidades clave

- Lógica de negocio y reglas de la aplicación
- Autenticación y autorización de usuarios
- Operaciones CRUD en bases de datos
- Exposición de APIs para el Front End
- Seguridad: encriptación, validación, CORS

Tecnologías de Back End más usadas



Python

Django, Flask. Popular en ciencia de datos y APIs REST.



Node.js

JavaScript en el servidor. Alta concurrencia. Express.js.



PHP

El más usado en hosting compartido. WordPress lo usa.



Java / C#

Empresariales, robustos. Spring Boot / ASP.NET.

Front End vs Back End

Aspecto	Front End	Back End
¿Dónde corre?	En el navegador del usuario	En el servidor remoto
Lo ve el usuario	✅ Sí — interfaz visible	❌ No — lógica oculta
Tecnologías	HTML, CSS, JavaScript	Python, Node.js, PHP, Java, C#
Enfoque principal	UX, UI, accesibilidad, responsivo	Seguridad, rendimiento, datos
Comunicación	Envía peticiones HTTP/API	Procesa y retorna respuestas
Ejemplos de tareas	Validar formulario, animar botón	Autenticar login, procesar pago

❓ Actividad: ¿"Mostrar un mensaje de error si la contraseña tiene menos de 8 caracteres" es Front End, Back End o ambos? Discute con tu compañero más cercano en el chat.

JSON y XML: El lenguaje de las APIs

JSON – JavaScript Object Notation

Formato ligero, legible y universal para el intercambio de datos entre sistemas. Es el estándar moderno para APIs REST.

```
{
  "usuario": {
    "id": 1042,
    "nombre": "María López",
    "correo": "maria@universidad.edu",
    "activo": true,
    "cursos": ["Prog2", "BD1", "Redes"]
  }
}
```

XML – eXtensible Markup Language

Más verboso que JSON, pero muy expresivo. Usado en servicios SOAP, configuraciones e industrias como salud y finanzas.

```
<usuario>
  <id>1042</id>
  <nombre>María López</nombre>
  <correo>maria@universidad.edu</correo>
  <activo>true</activo>
</usuario>
```

JSON vs XML – Comparativa

Característica	JSON	XML
Legibilidad	✓ Alta	⚠ Media
Tamaño del archivo	✓ Compacto	✗ Verboso
Velocidad de parseo	✓ Rápido	Más lento
Soporte de tipos	Básico	✓ Completo
Uso actual	✓ Dominante en APIs	Legado/empresarial

